Retrieval, Attention, and LLM

Programming Assignment 3

Aaryan Sharma
210110003

Anannay Jain
210110021

Aditya Kamble
25M0812

Shivansh Gupta
21D070067

CS 728 - Organization of Web information

Guide: Prof. Soumen Chakrabarti

Contents

1 Part 1: Classical Retrieval

1.1 Setup

We evaluate three retrieval methods on the test set of 5000 queries over 100 tools:

a) **BM25** – a sparse, term-frequency–based retrieval baseline using the Okapi BM25 scoring function. Tool descriptions are tokenized by whitespace, and query terms are matched against the tool corpus.

b) **msmarco-MiniLM** (`sentence-transformers/msmarco-MiniLM-L-6-v3`) – a lightweight dense retrieval model trained on the MS MARCO passage ranking dataset. Queries and tool descriptions are independently encoded into dense vectors, and cosine similarity is used for ranking.

c) **UAE-Large-V1** (`WhereIsAI/UAE-Large-V1`) – a larger dense retrieval model that produces high-quality universal sentence embeddings. Same encode-and-rank pipeline as above.

1.2 Results

Table 1: Part 1 – Classical Retrieval Results

Method	**Recall@1**	**Recall@5**
BM25	0.1860	0.3476
msmarco-MiniLM	0.3882	0.6106
UAE-Large-V1	0.6158	0.8690

1.3 Discussion

BM25 serves as a reasonable sparse baseline but is limited by its reliance on exact lexical overlap between queries and tool descriptions. The dense models substantially outperform BM25: msmarco-MiniLM roughly doubles Recall@1, while UAE-Large-V1 achieves 0.6158 Recall@1 and 0.8690 Recall@5. This confirms that semantic similarity captured by dense embeddings is far more effective than lexical matching for this tool-retrieval task, and that larger, more expressive encoder models yield further gains.

2 Part 2: Lost in the Middle

2.1 Setup

In this part, all 100 tool descriptions and the query are placed together in a single prompt and processed jointly by the Llama-3.2-1B-Instruct model. Rather than encoding queries and tools independently, we extract the model's internal attention matrices and use them as a retrieval signal.

Prompt structure. The prompt is constructed via the `PromptUtils` class. It begins with a user header and instruction ("Here are all the available tools:"), followed by all tool descriptions (in a randomly shuffled order), an instruction to output the correct `tool_id`, and finally the query. The assistant header is appended at the end.

Query span identification (`get_query_span`). The query tokens are the tokens that appear after the last tool description ends and before the end of the input sequence. Concretely, we set the query span as $(\max_j \text{end}_j,\ N)$ where end_j is the end index of tool j and N is the total sequence length.

Attention scoring (`query_to_docs_attention`). For each layer $\ell \in \{0, \ldots, L-1\}$, we first average the attention matrix across all heads to obtain a single $N \times N$ matrix. We then extract the rows corresponding to query tokens and, for each tool j with span (s_j, e_j), compute:

$$\text{score}_\ell(j) = \frac{1}{|Q|} \sum_{q \in Q} \sum_{t=s_j}^{e_j-1} A_\ell[q,t]$$

where Q is the set of query token positions and A_ℓ is the layer-averaged attention matrix. The final score for tool j is the average across all layers:

$$\text{score}(j) = \frac{1}{L} \sum_{\ell=0}^{L-1} \text{score}_\ell(j).$$

2.2 Results

Table 2: Part 2 – Attention-Based Retrieval Results (all heads, all layers)

Method	**Recall@1**	**Recall@5**
Attention-based (all heads)	0.0122	0.1620

The attention-based retrieval achieves significantly lower recall compared to the classical methods in Part 1. This is expected: averaging attention across *all* heads and layers dilutes the retrieval signal, since most heads serve syntactic, positional, or other non-retrieval functions.

2.3 Lost-in-the-Middle Analysis

Figure 1 shows the average attention score assigned to the gold (correct) tool as a function of its position in the prompt.

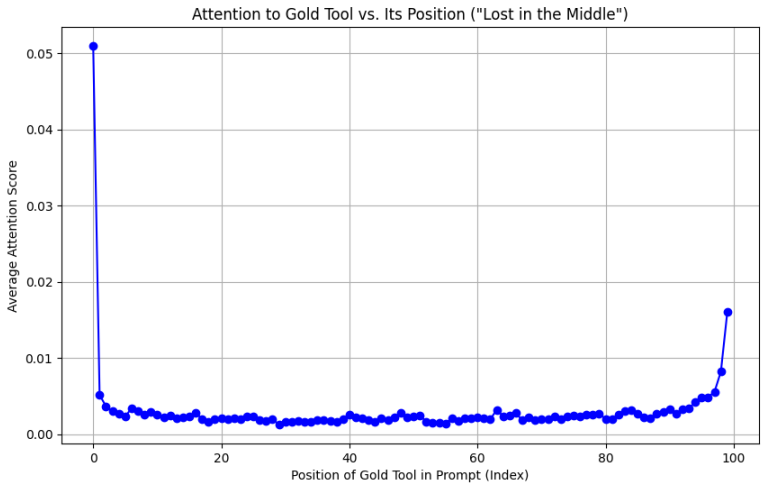

Figure 1: Average attention to the gold tool vs. its position in the prompt. The U-shaped curve demonstrates the "lost in the middle" phenomenon: tools placed at the very beginning or end of the prompt receive substantially higher attention, while tools in the middle are attended to much less.

The plot exhibits a clear U-shaped curve, confirming the **"lost in the middle"** phenomenon documented in prior work. Key observations:

- **Primacy effect**: Tools at position 0–2 receive the highest attention (~0.05), likely due to the model's tendency to attend strongly to tokens near the beginning of the context.
- **Recency effect**: Tools near the end (positions 95–99) also receive elevated attention (~0.008–0.016), because they are closest to the query tokens and thus benefit from local attention patterns.
- **Middle valley**: Tools in positions ~20–80 receive uniformly low attention (~0.002), making them harder to retrieve regardless of their relevance.

This positional bias is a fundamental limitation of using raw attention as a retrieval signal when all candidates are placed in a single long context.

3 Part 3: Retrieval Heads

3.1 Phase 1: Head Selection Strategy

The key insight motivating this part is that not all attention heads contribute equally to retrieval. A small subset of heads may act as "retrieval heads" that preferentially attend from query tokens to the relevant tool. By isolating these heads, we can obtain a cleaner retrieval signal.

Scoring via Mean Reciprocal Rank (MRR). We use the first 200 training queries to score each of the $L \times H$ attention heads (where $L = 16$ layers and $H = 32$ heads for Llama-3.2-1B). For each training query:

a) Construct the prompt (with shuffled tool order) and run the model to extract attention matrices.

b) For each head (l, h), compute a per-tool attention score: for each tool j, sum the attention from query tokens to tool-j tokens and average over query tokens:

$$s_{l,h}(j) = \frac{1}{|Q|} \sum_{q \in Q} \sum_{t=s_j}^{e_j-1} A_{l,h}[q, t]$$

c) Rank all tools by $s_{l,h}(\cdot)$ in descending order for head (l, h).

d) Find the rank r of the gold tool and add $\frac{1}{r+1}$ (reciprocal rank) to the cumulative score for head (l, h).

After processing all training queries, each head has an aggregated MRR score. We select the top $K = 20$ heads by this score.

This strategy identifies heads that *consistently* rank the correct tool highly, rather than heads that merely assign high absolute attention (which could be biased by position).

Selected heads (top 20). The selected 20 heads (as `(layer, head)` pairs) are:

```
[(6,11), (10,9), (10,1), (6,8), (5,11), (4,16), (5,10),
 (6,9), (2,20), (7,12), (8,19), (8,20), (10,23), (12,13),
 (11,22), (10,13), (13,20), (8,13), (5,9), (12,15)]
```

Notably, the selected heads are concentrated in the middle layers (layers 4–13), with layers 5, 6, 8, and 10 contributing multiple heads each. No heads from the first two layers or the last three layers were selected, suggesting that early layers handle low-level token processing and late layers handle generation rather than retrieval.

3.2 Phase 2: Retrieval Using Selected Heads

Scoring function At test time, we compute tool scores using *only* the selected heads. For each selected head (l, h):

$$s_{l,h}(j) = \frac{1}{|Q|} \sum_{q \in Q} \sum_{t=s_j}^{e_j-1} A_{l,h}[q, t]$$

The final score for tool j is the average across the K selected heads:

$$\text{score}(j) = \frac{1}{K} \sum_{(l,h) \in \mathcal{H}} s_{l,h}(j)$$

where $\mathcal{H}$ is the set of selected heads.

3.3 Results

Table 3: Part 3 – Retrieval Heads Results

Method	**Recall@1**	**Recall@5**
Selected heads ($K = 20$)	0.1550	0.5504

4 Comparison Across All Parts

Table 4: Summary of Recall across all methods

Part	**Method**	**Recall@1**	**Recall@5**
1	BM25	0.1860	0.3476
1	msmarco-MiniLM	0.3882	0.6106
1	UAE-Large-V1	0.6158	0.8690
2	Attention (all heads)	0.0122	0.1620
3	Selected heads ($K = 20$)	0.1550	0.5504